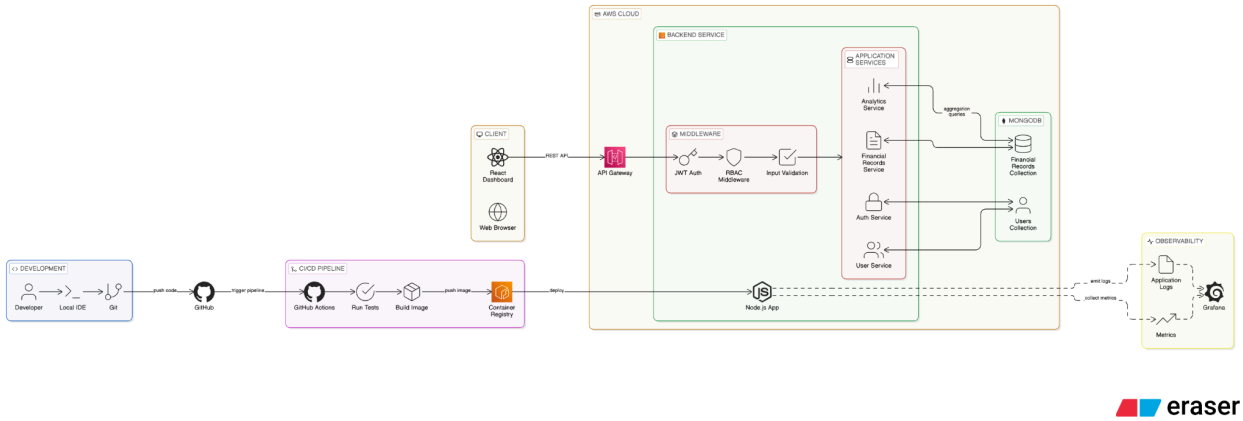

eraser

A Node.js & Express backend for a Finance Dashboard system supporting robust role-based access control, user management, and aggregated financial insights for front-end dashboards.

Features

- **User Authentication**: Secure JWT-based login and registration.
- **Role Management**: Hierarchical access consisting of *viewer*, *analyst*, and *admin* limits.
- **Financial Records**: Record creation, modifications, filters, and tracking.
- **Dashboard Summary**: Aggregate endpoints computing net balances, category breakdowns, and latest activity natively via MongoDB aggregation.

Setup Instructions

1. Install dependencies:

```shell
npm install
```

2. Create a *.env* file in the root folder with the following variables:

```
MONGO_URI=mongodb://localhost:27017/assignment_db
PORT=5000
JWT_SECRET=your_super_secret_jwt_key
```

3. Start the server:

```shell
npm run start
```

Database Documentation

The project uses **MongoDB** connected via *Mongoose*.

1. User Collection

Stores all registered users and their permission levels.

Field	Type	Description	Options / Defaults
_id	`ObjectId`	Unique identifier	*Auto-generated*
name	String	User's full name	-
email	String	User's email	`unique: true`
password	String	Hashed password	bcrypt hashed
role	String	Access level	`enum: ["viewer", "analyst", "admin"], default: "viewer"`
status	String	Account status	`enum: ["active", "inactive"], default: "active"`
timestamps	Dates	createdAt, updatedAt	*Auto-generated by Mongoose*

2. Record Collection

Stores transactions and financial entries.

Field	Type	Description	Options / Defaults
_id	`ObjectId`	Unique identifier	*Auto-generated*
amount	Number	Transaction amount	`required: true`
type	String	Flow direction	`enum: ["income", "expense"], required: true`
category	String	Identifier category	`required: true`
date	Date	Transaction date	`default: Date.now`
notes	String	Optional context	-
createdBy	`ObjectId`	Reference to User	`ref: "User", required: true`
timestamps	Dates	createdAt, updatedAt	*Auto-generated by Mongoose*

API Documentation

The Base URL for all endpoints is `http://localhost:5000` (or your `$PORT`).

Authentication APIs

1. Register User

- **Endpoint:** `POST /api/auth/register`
- **Access:** Public

- **Body:** `{ "name": "John Doe", "email": "john@test.com", "password": "password123" }`
- **Response:** `201 Created`

2. Login User

- **Endpoint:** `POST /api/auth/login`

- **Access:** Public
- **Body:** `{ "email": "john@test.com", "password": "password123" }`
- **Response:** `200 OK` (Returns `{ "token": "..." }`)

User Management APIs (Admin Only)

All requests require a valid JWT token sent in `Authorization: Bearer <token>`.

1. Get All Users

- **Endpoint:** `GET /api/users`
- **Access:** Admin
- **Response:** Array of user objects (excluding passwords).

2. Create User

- **Endpoint:** `POST /api/users`
- **Access:** Admin
- **Body:** `{ "name": "Analyst Bob", "email": "bob@test.com", "password": "secure123", "role": "analyst" }`
- **Response:** Returns newly created user (excluding password). Supports assigning `viewer`, `analyst`, or `admin` roles directly.

3. Update User Role

- **Endpoint:** `PUT /api/users/:id/role`
- **Access:** Admin
- **Body:** `{ "role": "admin" }` // viewer, analyst, admin
- **Response:** Returns updated user.

4. Update User Status

- **Endpoint:** `PUT /api/users/:id/status`
- **Access:** Admin
- **Body:** `{ "status": "inactive" }` // active, inactive
- **Response:** Returns updated user. (Inactive users are automatically denied authentication).

Record APIs

All requests require a valid JWT token.

1. Get Financial Dashboard Summary

- **Endpoint:** `GET /api/records/summary`
- **Access:** Viewer, Analyst, Admin (Viewers are restricted to *only* viewing this dashboard data).
- **Response:** Returns aggregated JSON object with:
  - `totalIncome`
  - `totalExpense`
  - `netBalance`
  - `categoryDistribution` (Array grouping totals by type and category)
  - `recentActivity` (Array of the 5 most recent records)

2. Create Record

- **Endpoint:** `POST /api/records`
- **Access:** Admin
- **Body:** `{ "amount": 5000, "type": "income", "category": "Salary", "notes": "March Pay" }`
- **Response:** Returns created record.

3. Get All Records

- **Endpoint:** `GET /api/records`
- **Access:** Analyst, Admin (Analysts can view individual records, Viewers cannot).
- **Query Parameters (Optional):**
  - `?type=income` (Filter by 'income' or 'expense')
  - `?category=Salary` (Filter by category)
  - `?startDate=2024-01-01&endDate=2024-01-31` (Filter by date ranges)
- **Response:** Sorted Array of Records (latest first).

4. Update Record

- **Endpoint:** `PUT /api/records/:id`
- **Access:** Admin
- **Body:** Fields to update (`amount`, `type`, `category`, `notes`).
- **Response:** Returns updated record.

5. Delete Record

- **Endpoint:** `DELETE /api/records/:id`
- **Access:** Admin
- **Response:** Success Message.